CS311 Operating Systems: Assignment 01 Mini Shell

Complete evaluation study guide + commented code + practice solutions

Purpose: Read it, compile the example programs, and practise editing the shell by hand.

Source documents: The uploaded **CS311 Operating Systems Assignment 01 - UPDATED - Mini Shell** and **OS Evaluation Reference Sheet**. The assignment specifies a prompt, `fork`/`exec`/`wait`, parent-side builtins, a custom process table and `ps`/`kill`, `&`, `<`, `>`, `>>`, pipes, `linenoise`, build tools, error checking, and debugging. The reference sheet additionally covers Make/CMake, libraries, gdb/Valgrind, environment functions, and shell internals.

Important caveat: These are **predicted practice questions**, not leaked or confirmed evaluation tasks. The included project is a learning implementation, not necessarily identical to your submitted files. The official **linenoise** source is not included; add `linenoise.c`/`.h` from the official project and enable its build switch to satisfy that specific assignment rule.

Fast start on Linux / WSL:

```
cd CS311_MiniShell_Eval_Prep/src
make clean && make
make test
./mysh
```

For official linenoise after adding `linenoise.c` and `linenoise.h` to `src/`:

```
make clean && make USE_LINENOISE=1
```

When you first study: start with Sections 1-3, then read the complete sources in Section 4, and finally solve the questions in Section 6 before checking the solutions.

1. One-page mental model

The shell is a loop: read input -> tokenize -> parse into commands -> run builtin in parent OR fork child -> apply file-descriptor wiring -> execvp -> foreground wait or background track -> prompt again.

The five operations you must never confuse:

- `fork()` makes a new child. In parent returns child's PID; in child returns `0`; on error `-1`.
- `execvp(file, argv)` replaces the running program; it **never returns on success**. `argv` ends in `NULL`.
- `waitpid(pid, &status, 0)` waits for a child; `waitpid(-1, &status, WNOHANG)` checks exited children without waiting.
- `pipe(fd)` creates `fd[0]` (READ) and `fd[1]` (WRITE).
- `dup2(old_fd, STDOUT_FILENO)` makes output go where old_fd goes. STDIN=0, STDOUT=1, STDERR=2.

Command examples to visualize:

```
ls -l                       -> fork + execvp + waitpid
cd /tmp                     -> chdir in PARENT (do not fork)
sleep 4 &                   -> fork, DON'T wait, record PID
ls > out.txt                -> child open(out, O_TRUNC), dup2(fd,1), execvp
ls >> out.txt               -> child open(out, O_APPEND), dup2(fd,1), execvp
wc -l < in.txt              -> child open(in, O_RDONLY), dup2(fd,0), execvp
cat in.txt | wc -l          -> pipe, 2 children, stdout child1 -> stdin child2
cat f | grep a | wc -l     -> 2 pipes, 3 children, close ALL unused ends
ps                          -> print your SHELL'S job table, not OS-wide ps
kill 12345                  -> validate/tracked PID, send SIGTERM, reap/remove
```

Why builtins run in parent: The state of `cd` and `exit` must persist in the shell. If `cd` runs in child, the parent stays in the same directory. In a pipeline/background, a builtin may run in child, without affecting parent state.

Very common evaluator traps: empty line; missing redirect filename; command not found; `argv` missing NULL; forgotten `close()` causing pipe hang; `kill` with bad PID; background zombies; `pwd > out` breaking later shell output; compiling without `-Wall`.

2. All assignment features: checklist and relevant code

Requirement	Function/file to inspect	What to know
Prompt/input	`src/main.c`	`linenoise()` or training fallback `getline()`; remove newline; clean up
Command arguments	`src/parser.c`	`tokenize`, `parse_segments`, NULL-terminated `argv`
Child execution	`src/exec.c`	`fork` -> `execvp` -> `_exit(127)` on failure
Foreground waiting	`src/exec.c`	`waitpid(pid, ..., 0)`, retry on `EINTR`
`pwd`, `cd`, `exit`	`src/builtins.c`	`getcwd`, `chdir`, `getenv(HOME)`
Own `ps` and `kill`	`src/builtins.c`	`Shell.jobs[]`, `strtol`, `kill(SIGTERM)`, `reap_jobs`
Background `&`	parser and exec	Mark segment background; don't wait; add jobs
`>` and `>>`	`redirect_stage`	`O_TRUNC` vs `O_APPEND`
`<`	`redirect_stage`	`O_RDONLY`, `dup2(fd, 0)`
Two or more pipes	`run_segment`	previous read end + new pipe; wait for every child
Multiple `&` segments	`parse_segments`	`sleep 2 & echo hi`
Make/CMake	`src/Makefile` / `src/CMakeLists.txt`	`-Wall`, `-g`, `$@`, `$<`, `$^`
Errors / resources	every file	check system-call returns; `free`, `close`, `waitpid`

Design limitation: This parser intentionally omits Bash quoting, escaping, wildcard expansion, variable expansion, `&&`, `||`, and `2>` because they are not requested by this assignment. Operators (`<`, `>`, `>>`, `|`, `&`) work even if not surrounded by spaces. `ps` reports tracked background child processes; it is not a system-wide process listing.

3. C and Linux quick-reference

```c
#include <unistd.h>     // fork, execvp, getpid, getppid, chdir, dup2, close, pipe
#include <sys/wait.h>   // waitpid, WNOHANG, WIFEXITED, WEXITSTATUS
#include <fcntl.h>      // open, O_RDONLY, O_CREAT, O_TRUNC, O_APPEND
#include <signal.h>     // kill, SIGTERM, SIGKILL
#include <string.h>     // strcmp, strtok_r, strlen, memset
#include <stdlib.h>     // getenv, setenv, unsetenv, malloc, free, strtol
#include <stdio.h>      // perror, printf, getline
#include <errno.h>      // errno, EINTR, ECHILD
```

- `strcmp(a,b) == 0` means the strings are equal. Never use `==` to compare string contents.
- `strtok_r()` modifies the string it parses; `saveptr` is needed for nested tokenization.
- `getline()` returns `-1` on EOF/error; its buffer must eventually be `free()`d.
- `getenv()` can return `NULL`; check before using it.
- `strtol()` should validate `errno`, end pointer, and nonnegative/valid PID range.
- `execvp` searches `PATH`; `execv` expects an executable path.
- `_exit()` is preferred after child exec failure; unlike `exit()`, it doesn't flush inherited buffered standard streams.
- `WIFEXITED(status)` must be true before using `WEXITSTATUS(status)`.
- `WIFSIGNALED(status)` reports signal-related termination; `WTERMSIG(status)` reads the signal number.

- Make sure every `malloc` is paired with `free`, every pipe/open FD with `close`, and children are eventually reaped.

Compile a single program: `gcc -std=c11 -Wall -Wextra -g q01_fork_exec_wait.c -o q01`.

gdb commands: `gdb ./mysh`, `break main`, `break run_segment`, `run`, `next`, `step`, `print pid`, `info locals`, `bt`, `continue`, `set follow-fork-mode child`.

Valgrind: `valgrind --leak-check=full ./mysh`.

4. Complete annotated Mini Shell implementation

The source below is **complete as a multi-file project**: copy each fenced block into its filename (or use the ready-to-compile files included in the ZIP). All main features are implemented and the source contains in-line comments explaining key operations.

Reading order: `shell.h` -> `main.c` -> `parser.c` -> `builtins.c` -> `exec.c` -> `Makefile` -> `CMakeLists.txt`.

shell.h

File: `src/shell.h`

```c
#ifndef SHELL_H
#define SHELL_H

// Shared structures and function declarations for our mini shell.
// Build: make  (getline mode; see README to enable official linenoise)

#include <stdbool.h>
#include <stddef.h>
#include <sys/types.h>

#define MAX_ARGS 64
#define MAX_STAGES 16
#define MAX_SEGMENTS 32
#define MAX_TOKENS 256
#define MAX_JOBS 256

// One command between pipes, e.g. "grep x < in.txt".
typedef struct {
    char *argv[MAX_ARGS];         // NULL-terminated arguments for execvp()
    int argc;                     // Number of actual arguments
    char *infile;                 // Filename after '<'; NULL when unused
    char *outfile;                // Filename after '>' or '>>'
    bool append;                  // true means >>, false means >
} Stage;

// One pipeline; one input line may contain multiple segments separated by '&'.
typedef struct {
    Stage stages[MAX_STAGES];
    int count;                    // Number of stages in pipeline
    bool background;              // True when segment ends with '&'
} Segment;

// Tokens own their allocated string memory. A Stage points into these strings.
typedef struct {
    char *items[MAX_TOKENS];
    int count;
```

```
} Tokens;

typedef struct {
    pid_t pid;
    char name[128];
} Job;

typedef struct {
    Job jobs[MAX_JOBS];
    int job_count;
    bool should_exit;
} Shell;

// parser.c
int tokenize(const char *line, Tokens *tokens);
void free_tokens(Tokens *tokens);
int parse_segments(const Tokens *tokens, Segment segments[], int *count);

// builtins.c
bool is_builtin(const char *name);
int run_builtin(Shell *shell, const Stage *stage, bool in_child);
int add_job(Shell *shell, pid_t pid, const char *name);
void remove_job(Shell *shell, pid_t pid);
void reap_jobs(Shell *shell);

// exec.c
int redirect_stage(const Stage *stage);
int run_segment(Shell *shell, const Segment *segment);

#endif
```

main.c

File: `src/main.c`

```
#define _POSIX_C_SOURCE 200809L
#include "shell.h"
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#ifdef USE_LINENOISE
#include "linenoise.h" // Official library: https://github.com/antirez/linenoise
#endif

int main(void) {
    Shell shell = {0}; // Job table starts empty.
    Segment segments[MAX_SEGMENTS];

#ifndef USE_LINENOISE
    char *buffer = NULL; // getline() allocates/reallocates this.
    size_t capacity = 0;
#endif

    while (!shell.should_exit) {
```

```
        reap_jobs(&shell); // Remove any finished background processes.

#ifdef USE_LINENOISE
        // Assignment 01 specifically requests the official linenoise library.
        char *line = linenoise("mysh> ");
        if (line == NULL) break; // Ctrl-D / end of input.
        if (*line != '\0') linenoiseHistoryAdd(line);
#else
        // Portable training fallback for systems without downloaded linenoise.
        if (isatty(STDIN_FILENO)) {
            printf("mysh> ");
            fflush(stdout);
        }
        ssize_t n = getline(&buffer, &capacity, stdin);
        if (n == -1) {
            if (ferror(stdin) && errno != EINTR)
                perror("getline");
            break;
        }
        buffer[strcspn(buffer, "\n")] = '\0';
        char *line = buffer;
#endif

        Tokens tokens;
        int command_count = 0;
        if (tokenize(line, &tokens) == 0) {
            if (parse_segments(&tokens, segments, &command_count) == 0) {
                for (int i = 0; i < command_count; ++i) {
                    run_segment(&shell, &segments[i]);
                    if (shell.should_exit) break;
                }
            }
            free_tokens(&tokens); // Pointers in Stage expire here.
        }

#ifdef USE_LINENOISE
        linenoiseFree(line);
#endif
    }

#ifndef USE_LINENOISE
    free(buffer);
#endif
    // The shell is exiting. A production shell may decide to signal/wait for
    // outstanding background jobs; this A01 version lets them continue.
    return 0;
}
```

parser.c

File: `src/parser.c`

```
#define _POSIX_C_SOURCE 200809L
#include "shell.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

```c
#include <ctype.h>

// Return 1 for a shell operator. Longest match matters: '>>' is ONE token.
static bool is_operator(const char *s) {
    return strcmp(s, "&") == 0 || strcmp(s, "|") == 0 ||
           strcmp(s, "<") == 0 || strcmp(s, ">") == 0 ||
           strcmp(s, ">>") == 0;
}

// Duplicate n bytes into heap memory and add a terminating '\0'.
static char *copy_part(const char *p, size_t n) {
    char *out = malloc(n + 1);
    if (out == NULL) return NULL;
    memcpy(out, p, n);
    out[n] = '\0';
    return out;
}

void free_tokens(Tokens *tokens) {
    for (int i = 0; i < tokens->count; ++i)
        free(tokens->items[i]);
    tokens->count = 0;
}

// Turns:  cat<in.txt|wc -l>>out.txt
// into:  cat < in.txt | wc -l >> out.txt
// No quoting, wildcard expansion, variables, or escaping: not required by A01.
int tokenize(const char *line, Tokens *tokens) {
    memset(tokens, 0, sizeof(*tokens));
    const char *p = line;

    while (*p != '\0') {
        if (isspace((unsigned char)*p)) { ++p; continue; }
        if (tokens->count >= MAX_TOKENS) {
            fprintf(stderr, "Too many tokens (max %d)\n", MAX_TOKENS);
            goto error;
        }

        const char *start = p;
        size_t n;
        if (*p == '>' && p[1] == '>') {
            p += 2; n = 2;                  // append operator
        } else if (strchr("<>|&", *p) != NULL) {
            ++p; n = 1;                     // single-character operator
        } else {
            while (*p && !isspace((unsigned char)*p) &&
                   strchr("<>|&", *p) == NULL)
                ++p;
            n = (size_t)(p - start);
        }
        tokens->items[tokens->count] = copy_part(start, n);
        if (tokens->items[tokens->count] == NULL) {
            perror("malloc");
            goto error;
        }
        tokens->count++;
```

```
    }
    return 0;
error:
    free_tokens(tokens);
    return -1;
}

// Each Stage has at least one executable word. Redirection filenames do not
// appear inside argv. Empty pipelines such as "ls | | wc" are rejected.
int parse_segments(const Tokens *tokens, Segment segments[], int *count) {
    *count = 0;
    if (tokens->count == 0) return 0;     // Empty line is okay.

    memset(segments, 0, sizeof(*segments) * MAX_SEGMENTS);
    int seg = 0, st = 0;
    segments[seg].count = 1;

    for (int i = 0; i < tokens->count; ++i) {
        const char *t = tokens->items[i];
        Stage *s = &segments[seg].stages[st];

        if (strcmp(t, "|") == 0) {
            if (s->argc == 0) goto bad_syntax;
            if (st + 1 >= MAX_STAGES) {
                fprintf(stderr, "Too many pipe stages\n");
                return -1;
            }
            st++;
            segments[seg].count++;
        } else if (strcmp(t, "&") == 0) {
            if (s->argc == 0) goto bad_syntax;
            segments[seg].background = true;
            seg++;
            if (seg >= MAX_SEGMENTS) {
                fprintf(stderr, "Too many commands separated by &\n");
                return -1;
            }
            st = 0;
            if (i != tokens->count - 1)
                segments[seg].count = 1;
        } else if (strcmp(t, "<") == 0 ||
                   strcmp(t, ">") == 0 ||
                   strcmp(t, ">>") == 0) {
            if (i + 1 >= tokens->count ||
                is_operator(tokens->items[i + 1])) {
                fprintf(stderr, "Missing filename after %s\n", t);
                return -1;
            }
            char *file = tokens->items[++i];
            if (strcmp(t, "<") == 0) {
                s->infile = file;
            } else {
                s->outfile = file;
                s->append = (strcmp(t, ">>") == 0);
            }
        } else {
```

```
            if (s->argc >= MAX_ARGS - 1) {
                fprintf(stderr, "Too many command arguments\n");
                return -1;
            }
            s->argv[s->argc++] = tokens->items[i];
            s->argv[s->argc] = NULL; // required terminator for execvp
        }
    }
    if (tokens->count > 0 &&
        strcmp(tokens->items[tokens->count - 1], "&") == 0) {
        *count = seg; // All segments ended with '&'; no empty final one.
    } else {
        if (segments[seg].stages[st].argc == 0) goto bad_syntax;
        *count = seg + 1;
    }
    return 0;

bad_syntax:
    fprintf(stderr, "Invalid command syntax (missing command near | or &)\n");
    return -1;
}
```

builtins.c

File: `src/builtins.c`

```
#define _GNU_SOURCE
#include "shell.h"
#include <errno.h>
#include <limits.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/wait.h>
#include <unistd.h>

bool is_builtin(const char *name) {
    if (name == NULL) return false;
    return strcmp(name, "cd") == 0 || strcmp(name, "pwd") == 0 ||
           strcmp(name, "exit") == 0 || strcmp(name, "ps") == 0 ||
           strcmp(name, "kill") == 0;
}

int add_job(Shell *shell, pid_t pid, const char *name) {
    if (shell->job_count >= MAX_JOBS) {
        fprintf(stderr, "Background job table is full\n");
        return -1;
    }
    Job *j = &shell->jobs[shell->job_count++];
    j->pid = pid;
    snprintf(j->name, sizeof(j->name), "%s", name);
    return 0;
}

void remove_job(Shell *shell, pid_t pid) {
    for (int i = 0; i < shell->job_count; ++i) {
```

```
        if (shell->jobs[i].pid == pid) {
            // Replace with last entry: O(1) removal, job order can change.
            shell->jobs[i] = shell->jobs[shell->job_count - 1];
            shell->job_count--;
            return;
        }
    }
}

// Non-blocking cleanup: a finished background child otherwise becomes zombie.
void reap_jobs(Shell *shell) {
    int status;
    pid_t done;
    while ((done = waitpid(-1, &status, WNOHANG)) > 0) {
        remove_job(shell, done);
    }
    if (done == -1 && errno != ECHILD)
        perror("waitpid WNOHANG");
}

static bool tracked_pid(const Shell *shell, pid_t pid) {
    for (int i = 0; i < shell->job_count; ++i)
        if (shell->jobs[i].pid == pid) return true;
    return false;
}

// Called for a shell builtin (normally in the parent). When inside a pipeline
// or background command, it runs in a CHILD so it cannot change parent state.
int run_builtin(Shell *shell, const Stage *s, bool in_child) {
    const char *cmd = s->argv[0];
    if (!is_builtin(cmd)) return -1; // -1 means "not a builtin"

    if (strcmp(cmd, "pwd") == 0) {
        if (s->argc != 1) {
            fprintf(stderr, "pwd: no arguments expected\n");
            return 1;
        }
        char *cwd = getcwd(NULL, 0); // GNU extension allocates enough space
        if (cwd == NULL) { perror("pwd"); return 1; }
        printf("%s\n", cwd);
        free(cwd);
        return 0;
    }

    if (strcmp(cmd, "cd") == 0) {
        if (s->argc > 2) {
            fprintf(stderr, "cd: too many arguments\n");
            return 1;
        }
        const char *path = (s->argc == 2) ? s->argv[1] : getenv("HOME");
        if (path == NULL) {
            fprintf(stderr, "cd: HOME is not set\n");
            return 1;
        }
        if (chdir(path) == -1) { perror("cd"); return 1; }
        return 0;
```

```
}

if (strcmp(cmd, "exit") == 0) {
    if (s->argc != 1) {
        fprintf(stderr, "exit: no arguments expected\n");
        return 1;
    }
    if (!in_child) shell->should_exit = true;
    return 0;
}

if (strcmp(cmd, "ps") == 0) {
    if (s->argc != 1) {
        fprintf(stderr, "ps: no arguments expected\n");
        return 1;
    }
    if (!in_child) reap_jobs(shell);
    printf("PID\tNAME\n");
    for (int i = 0; i < shell->job_count; ++i) {
        printf("%ld\t%s\n", (long)shell->jobs[i].pid,
               shell->jobs[i].name);
    }
    return 0;
}

// This assignment's kill only targets PIDs tracked by our mini shell.
if (strcmp(cmd, "kill") == 0) {
    if (s->argc != 2) {
        fprintf(stderr, "usage: kill PID\n");
        return 1;
    }
    char *end = NULL;
    errno = 0;
    long raw = strtol(s->argv[1], &end, 10);
    if (errno != 0 || end == s->argv[1] || *end != '\0' ||
        raw <= 0 || raw > INT_MAX) {
        fprintf(stderr, "kill: invalid PID\n");
        return 1;
    }
    pid_t target = (pid_t)raw;
    if (!tracked_pid(shell, target)) {
        fprintf(stderr, "kill: PID not tracked by mysh\n");
        return 1;
    }
    if (kill(target, SIGTERM) == -1) {
        if (errno == ESRCH) {
            // Might have exited between the lookup and signal.
            if (!in_child) reap_jobs(shell);
        }
        perror("kill");
        return 1;
    }
    // Reap the child; this can block if a process ignores SIGTERM.
    // No waitpid in pipeline child: it is NOT parent of tracked jobs.
    if (!in_child) {
        int st;
```

```
            pid_t done;
            do { done = waitpid(target, &st, 0); }
            while (done == -1 && errno == EINTR);
            if (done == -1 && errno != ECHILD) {
                perror("waitpid kill");
                return 1;
            }
            remove_job(shell, target);
        }
        return 0;
    }
    return 1;
}
```

exec.c

File: `src/exec.c`

```
#define _POSIX_C_SOURCE 200809L
#include "shell.h"
#include <errno.h>
#include <fcntl.h>
#include <signal.h>
#include <stdio.h>
#include <sys/wait.h>
#include <unistd.h>

// Change standard input/output BEFORE execvp(). Called in a child, OR
// temporarily in parent while executing a builtin (then restore descriptors).
int redirect_stage(const Stage *s) {
    if (s->infile != NULL) {
        int fd = open(s->infile, O_RDONLY);
        if (fd == -1) { perror(s->infile); return -1; }
        if (dup2(fd, STDIN_FILENO) == -1) {
            perror("dup2 stdin");
            close(fd);
            return -1;
        }
        if (close(fd) == -1) perror("close input");
    }

    if (s->outfile != NULL) {
        int flags = O_WRONLY | O_CREAT | (s->append ? O_APPEND : O_TRUNC);
        int fd = open(s->outfile, flags, 0644);
        if (fd == -1) { perror(s->outfile); return -1; }
        if (dup2(fd, STDOUT_FILENO) == -1) {
            perror("dup2 stdout");
            close(fd);
            return -1;
        }
        if (close(fd) == -1) perror("close output");
    }
    return 0;
}

static void wait_for_child(pid_t pid) {
    int status;
```

```
    pid_t result;
    do {
        result = waitpid(pid, &status, 0);
    } while (result == -1 && errno == EINTR);

    if (result == -1) perror("waitpid");
    // Optional: inspect WIFEXITED(status) and WEXITSTATUS(status).
}

// A builtin that changes the shell's state (cd/exit) MUST run in the parent.
// Save and restore stdin/stdout if redirection was requested for the builtin.
static int run_parent_builtin(Shell *shell, const Stage *s) {
    int saved_in = -1, saved_out = -1, result = 1;
    bool use_in = s->infile != NULL;
    bool use_out = s->outfile != NULL;

    fflush(stdout); // Flush before changing stdout's destination.
    if (use_in) {
        saved_in = dup(STDIN_FILENO);
        if (saved_in == -1) { perror("dup stdin"); goto cleanup; }
    }
    if (use_out) {
        saved_out = dup(STDOUT_FILENO);
        if (saved_out == -1) { perror("dup stdout"); goto cleanup; }
    }
    if (redirect_stage(s) == -1) goto cleanup;

    result = run_builtin(shell, s, false);
    fflush(stdout); // Flush builtin output BEFORE restoring stdout.

cleanup:
    if (saved_in != -1) {
        if (dup2(saved_in, STDIN_FILENO) == -1) {
            perror("restore stdin"); result = 1;
        }
        if (close(saved_in) == -1) perror("close saved stdin");
    }
    if (saved_out != -1) {
        if (dup2(saved_out, STDOUT_FILENO) == -1) {
            perror("restore stdout"); result = 1;
        }
        if (close(saved_out) == -1) perror("close saved stdout");
    }
    return result;
}

int run_segment(Shell *shell, const Segment *segment) {
    if (segment->count <= 0) return 0;

    // Foreground single-stage builtins run in parent; all other cases fork.
    const Stage *first = &segment->stages[0];
    if (segment->count == 1 && !segment->background &&
        is_builtin(first->argv[0])) {
        return run_parent_builtin(shell, first);
    }
```

```
// Don't exceed the internal table before starting background children.
if (segment->background &&
    shell->job_count + segment->count > MAX_JOBS) {
    fprintf(stderr, "Not enough room in background job table\n");
    return 1;
}

pid_t pids[MAX_STAGES];
int spawned = 0;
int prev_read = -1; // Read end of the previous pipe, or none.

fflush(stdout);     // Avoid duplicate output after fork.

for (int i = 0; i < segment->count; ++i) {
    int p[2] = {-1, -1};
    const bool last = (i == segment->count - 1);

    if (!last && pipe(p) == -1) {
        perror("pipe");
        goto fork_failure;
    }

    pid_t pid = fork();
    if (pid == -1) {
        perror("fork");
        if (p[0] != -1) close(p[0]);
        if (p[1] != -1) close(p[1]);
        goto fork_failure;
    }

    if (pid == 0) {
        // CHILD: pipe connections are made first.
        if (prev_read != -1 &&
            dup2(prev_read, STDIN_FILENO) == -1) {
            perror("pipe stdin"); _exit(1);
        }
        if (!last && dup2(p[1], STDOUT_FILENO) == -1) {
            perror("pipe stdout"); _exit(1);
        }

        // Close original descriptors after dup2().
        if (prev_read != -1) close(prev_read);
        if (p[0] != -1) close(p[0]);
        if (p[1] != -1) close(p[1]);

        // Explicit <, >, >> take priority over pipe endpoints.
        const Stage *s = &segment->stages[i];
        if (redirect_stage(s) == -1) _exit(1);

        if (is_builtin(s->argv[0])) {
            int code = run_builtin(shell, s, true);
            fflush(stdout); // _exit doesn't flush stdio buffers.
            _exit(code == 0 ? 0 : 1);
        }

        execvp(s->argv[0], s->argv);
```

```
            perror(s->argv[0]); // execvp returns ONLY upon failure.
            _exit(127);
        }

        // PARENT: keep PIDs, close unneeded write end immediately.
        pids[spawned++] = pid;
        if (prev_read != -1 && close(prev_read) == -1)
            perror("close old pipe read end");
        if (p[1] != -1 && close(p[1]) == -1)
            perror("close pipe write end");
        prev_read = p[0];
    }

    if (prev_read != -1 && close(prev_read) == -1)
        perror("close final read end");

    if (segment->background) {
        // All pipeline processes are tracked, not just the last one.
        for (int i = 0; i < spawned; ++i)
            add_job(shell, pids[i], segment->stages[i].argv[0]);
    } else {
        for (int i = 0; i < spawned; ++i)
            wait_for_child(pids[i]);
    }
    return 0;

fork_failure:
    if (prev_read != -1) close(prev_read);
    // If a pipeline only partly started, stop children and collect statuses.
    for (int i = 0; i < spawned; ++i) {
        if (kill(pids[i], SIGTERM) == -1 && errno != ESRCH)
            perror("kill after failed fork");
    }
    for (int i = 0; i < spawned; ++i)
        wait_for_child(pids[i]);
    return 1;
}
```

Makefile

File: `src/Makefile`

```
# CS311 Mini Shell
# Default make uses getline(), so the project runs without external downloads.
# To satisfy the linenoise requirement, copy official linenoise.c/.h here, then:
#     make clean && make USE_LINENOISE=1

CC = gcc
CFLAGS = -std=c11 -Wall -Wextra -g
OBJS = main.o parser.o exec.o builtins.o

ifeq ($(USE_LINENOISE),1)
CFLAGS += -DUSE_LINENOISE
OBJS += linenoise.o
endif

mysh: $(OBJS)
```

```
    $(CC) $(CFLAGS) -o $@ $^

%.o: %.c shell.h
    $(CC) $(CFLAGS) -c $< -o $@

.PHONY: clean test

clean:
    rm -f mysh *.o

test: mysh
    bash ../tests/run_tests.sh
```

CMakeLists.txt

File: `src/CMakeLists.txt`

```
cmake_minimum_required(VERSION 3.10)
project(mysh C)
set(CMAKE_C_STANDARD 11)
add_executable(mysh main.c parser.c exec.c builtins.c)
target_compile_options(mysh PRIVATE -Wall -Wextra -g)
# After adding the official library files, enable with:
# cmake -S . -B build -DUSE_LINENOISE=ON
option(USE_LINENOISE "Build with official linenoise library" OFF)
if(USE_LINENOISE)
    target_sources(mysh PRIVATE linenoise.c)
    target_compile_definitions(mysh PRIVATE USE_LINENOISE)
endif()
```

5. Commands to test in your own lab

Type these into `./mysh`, not the normal Bash shell. The actual PIDs vary.

```
pwd
cd /tmp
pwd
cd
pwd
ls
ls -l
invalid_command_123

echo hello > a.txt
echo world >> a.txt
cat < a.txt
wc -l < a.txt
seq 10 | wc -l
seq 10 | head -n 4 | wc -l
seq 10 | wc -l > count.txt
cat count.txt
sleep 30 &
ps
kill PID_FROM_PS
ps
exit
```

Check for helpful errors: `cat <`, `ls >`, `ls |`, `| wc`, `kill xyz`, `cd /does/not/exist`, and `missing_command`. A robust shell should print an error rather than crash.

Special test: `pwd > where.txt` followed by `pwd` should redirect only the first output; the second `pwd` must display on the terminal again.

Rebuild and test after edits:

```
cd src
make clean && make
make test
```

6. Predicted lab evaluation questions - WITH COMPLETE SOLUTIONS

These are representative tasks the instructor **could** ask based on the uploaded assignment/reference sheet, NOT confirmed questions. For every question: read the statement, try writing code from scratch, then compare with the worked solution. Each example has a standalone `.c` file in `practice/`.

Q1 - Fork, execvp, waitpid

Possible task: Create one child. In the child show its PID and run `ls -l`. The parent waits and reports the exit status.

Idea: Use `fork()`, `execvp()`, a `NULL`-terminated argv, and `waitpid()`. Failure in exec should call `_exit(127)`.

Compile and run:

```
gcc -Wall -Wextra -g practice/q01_fork_exec_wait.c -o /tmp/practice_eval && /tmp/practice_eval
```

File: `practice/q01_fork_exec_wait.c`

```c
// Mock Q1: create a child; child runs "ls -l"; parent waits.
// gcc -Wall -Wextra -g q01_fork_exec_wait.c -o q01 && ./q01
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <errno.h>

int main(void) {
    pid_t pid = fork();
    if (pid == -1) { perror("fork"); return 1; }

    if (pid == 0) {                         // Child branch
        printf("Child PID = %ld\n", (long)getpid());
        fflush(stdout);                    // exec may replace stdout buffers
        char *args[] = {"ls", "-l", NULL}; // execvp needs NULL at end
        execvp(args[0], args);              // Looks up ls in PATH
        perror("execvp");                 // Runs only if execvp fails
        _exit(127);                         // Do not resume shell code
    }

    int status;
    pid_t done;
    do { done = waitpid(pid, &status, 0); }
    while (done == -1 && errno == EINTR);
    if (done == -1) { perror("waitpid"); return 1; }
```

```
    if (WIFEXITED(status))
        printf("Child finished, exit status %d\n", WEXITSTATUS(status));
    else
        printf("Child finished abnormally\n");
    return 0;
}
```

Q2 - Parent PID vs child PID

Possible task: Demonstrate the values returned from `fork()` and print parent/child PIDs.

Idea: Child receives 0 from fork; parent receives the child PID; process order can vary.

Compile and run:

```
gcc -Wall -Wextra -g practice/q02_fork_count.c -o /tmp/practice_eval && /tmp/practice_eval
```

File: `practice/q02_fork_count.c`

```
// Mock Q2: distinguish parent PID, child PID, and fork return value.
// gcc -Wall -Wextra q02_fork_count.c -o q02 && ./q02
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void) {
    pid_t pid = fork();
    if (pid == -1) { perror("fork"); return 1; }
    if (pid == 0) {
        printf("CHILD: my PID=%ld, parent PID=%ld, fork result=%ld\n",
               (long)getpid(), (long)getppid(), (long)pid);
        return 0;
    }
    printf("PARENT: my PID=%ld, child's PID=%ld\n",
           (long)getpid(), (long)pid);
    if (waitpid(pid, NULL, 0) == -1) { perror("waitpid"); return 1; }
    return 0;
}
```

Q3 - Output redirection

Possible task: Run `ls -l` in a child, saving output to `result.txt` with overwrite mode.

Idea: `open(O_WRONLY|O_CREAT|O_TRUNC, 0644)` then `dup2(fd, STDOUT_FILENO)`, close, exec.

Compile and run:

```
gcc -Wall -Wextra -g practice/q03_redirect.c -o /tmp/practice_eval && /tmp/practice_eval
```

File: `practice/q03_redirect.c`

```
// Mock Q3: redirect an external command's stdout to result.txt.
// gcc -Wall -Wextra q03_redirect.c -o q03 && ./q03
#include <fcntl.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
int main(void) {
    pid_t pid = fork();
    if (pid == -1) { perror("fork"); return 1; }
    if (pid == 0) {
        int fd = open("result.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
```

```
        if (fd == -1) { perror("open"); _exit(1); }
        if (dup2(fd, STDOUT_FILENO) == -1) {
            perror("dup2"); close(fd); _exit(1);
        }
        if (close(fd) == -1) { perror("close"); _exit(1); }
        char *args[] = {"ls", "-l", NULL};
        execvp(args[0], args);
        perror("execvp"); _exit(127);
    }
    if (waitpid(pid, NULL, 0) == -1) { perror("waitpid"); return 1; }
    puts("Check result.txt for redirected output.");
    return 0;
}
```

Q4 - Two-command pipeline

Possible task: Implement `seq 10 | wc -l` using one pipe and two children.

Idea: Child A stdout -> pipe write; Child B stdin <- pipe read; parent closes both ends.

Compile and run:

```
gcc -Wall -Wextra -g practice/q04_two_pipes.c -o /tmp/practice_eval && /tmp/practice_eval
```

File: `practice/q04_two_pipes.c`

```
// Mock Q4: implement "seq 10 | wc -l" using one pipe and two children.
// gcc -Wall -Wextra q04_two_pipes.c -o q04 && ./q04
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void) {
    int p[2];
    if (pipe(p) == -1) { perror("pipe"); return 1; }
    pid_t a = fork();
    if (a == -1) { perror("fork A"); close(p[0]); close(p[1]); return 1; }
    if (a == 0) {
        if (dup2(p[1], STDOUT_FILENO) == -1) { perror("dup2 A"); _exit(1); }
        close(p[0]); close(p[1]);
        char *args[] = {"seq", "10", NULL};
        execvp(args[0], args); perror("seq"); _exit(127);
    }
    pid_t b = fork();
    if (b == -1) {
        perror("fork B"); close(p[0]); close(p[1]);
        waitpid(a, NULL, 0); return 1;
    }
    if (b == 0) {
        if (dup2(p[0], STDIN_FILENO) == -1) { perror("dup2 B"); _exit(1); }
        close(p[0]); close(p[1]);
        char *args[] = {"wc", "-l", NULL};
        execvp(args[0], args); perror("wc"); _exit(127);
    }
    close(p[0]); close(p[1]); // Parent MUST close both ends.
    if (waitpid(a, NULL, 0) == -1) perror("waitpid A");
    if (waitpid(b, NULL, 0) == -1) perror("waitpid B");
    return 0;
}
```

Q5 - Three-command pipeline

Possible task: Implement `seq 10 | head -n 4 | wc -l`.

Idea: For three commands there are two pipes. Child i connects previous input and next output.

Compile and run:

```
gcc -Wall -Wextra -g practice/q05_three_commands.c -o /tmp/practice_eval && /tmp/practice_eval
```

File: `practice/q05_three_commands.c`

```c
// Mock Q5: implement "seq 10 | head -n 4 | wc -l" with 3 children.
// gcc -Wall -Wextra q05_three_commands.c -o q05 && ./q05
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void) {
    int pipes[2][2];
    if (pipe(pipes[0]) == -1) { perror("pipe 0"); return 1; }
    if (pipe(pipes[1]) == -1) {
        perror("pipe 1"); close(pipes[0][0]); close(pipes[0][1]); return 1;
    }
    char *cmd1[] = {"seq", "10", NULL};
    char *cmd2[] = {"head", "-n", "4", NULL};
    char *cmd3[] = {"wc", "-l", NULL};
    char **cmds[3] = {cmd1, cmd2, cmd3};
    pid_t children[3];
    int started = 0;
    for (int i = 0; i < 3; ++i) {
        pid_t pid = fork();
        if (pid == -1) { perror("fork"); break; }
        if (pid == 0) {
            if (i > 0 && dup2(pipes[i - 1][0], STDIN_FILENO) == -1)
                { perror("dup2 in"); _exit(1); }
            if (i < 2 && dup2(pipes[i][1], STDOUT_FILENO) == -1)
                { perror("dup2 out"); _exit(1); }
            for (int k = 0; k < 2; ++k) {
                close(pipes[k][0]); close(pipes[k][1]);
            }
            execvp(cmds[i][0], cmds[i]);
            perror("execvp"); _exit(127);
        }
        children[started++] = pid;
    }
    // Parent closes ALL pipe descriptors; otherwise wc may wait for EOF.
    for (int k = 0; k < 2; ++k) {
        close(pipes[k][0]); close(pipes[k][1]);
    }
    for (int i = 0; i < started; ++i)
        if (waitpid(children[i], NULL, 0) == -1) perror("waitpid");
    return started == 3 ? 0 : 1;
}
```

Q6 - Background processes

Possible task: Launch `sleep 2` without blocking the parent. Check for completion with WNOHANG.

Idea: Parent does not immediately wait; it eventually reaps the child to avoid a zombie.

Compile and run:

```
gcc -Wall -Wextra -g practice/q06_background.c -o /tmp/practice_eval && /tmp/practice_eval
```

File: `practice/q06_background.c`

```c
// Mock Q6: launch sleep 2 in background; periodically reap with WNOHANG.
// gcc -Wall -Wextra q06_background.c -o q06 && ./q06
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <errno.h>
int main(void) {
    pid_t pid = fork();
    if (pid == -1) { perror("fork"); return 1; }
    if (pid == 0) {
        char *args[] = {"sleep", "2", NULL};
        execvp(args[0], args);
        perror("execvp"); _exit(127);
    }
    printf("Prompt would return immediately (background PID %ld)\n", (long)pid);
    for (;;) {
        int status;
        pid_t done = waitpid(pid, &status, WNOHANG);
        if (done == -1) {
            if (errno == EINTR) continue;
            perror("waitpid"); return 1;
        }
        if (done == pid) {
            puts("Child has finished and is reaped.");
            break;
        }
        puts("Child still running; shell can do other work.");
        sleep(1); // Demo only: real shell instead reads the next command.
    }
    return 0;
}
```

Q7 - Environment variables

Possible task: Read HOME, define OS_EVAL=READY, read it, then unset it.

Idea: Know `getenv()`, `setenv()`, and `unsetenv()`; check for NULL/error.

Compile and run:

```
gcc -Wall -Wextra -g practice/q07_env.c -o /tmp/practice_eval && /tmp/practice_eval
```

File: `practice/q07_env.c`

```c
// Mock Q7: use getenv, setenv, unsetenv and check errors.
// gcc -Wall -Wextra q07_env.c -o q07 && ./q07
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
int main(void) {
    const char *home = getenv("HOME");
    printf("HOME=%s\n", home != NULL ? home : "(not set)");
    if (setenv("OS_EVAL", "READY", 1) == -1) {
```

```
        perror("setenv"); return 1;
    }
    printf("OS_EVAL=%s\n", getenv("OS_EVAL"));
    if (unsetenv("OS_EVAL") == -1) {
        perror("unsetenv"); return 1;
    }
    puts("OS_EVAL removed");
    return 0;
}
```

Q8 - Wait status

Possible task: Make a child exit with code 42. Have the parent print that code.

Idea: Use WIFEXITED before WEXITSTATUS.

Compile and run:

```
gcc -Wall -Wextra -g practice/q08_wait_status.c -o /tmp/practice_eval && /tmp/practice_eval
```

File: `practice/q08_wait_status.c`

```c
// Mock Q8: child exits with status 42; parent reads it with waitpid macros.
// gcc -Wall -Wextra q08_wait_status.c -o q08 && ./q08
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(void) {
    pid_t p = fork();
    if (p == -1) { perror("fork"); return 1; }
    if (p == 0) _exit(42);
    int status;
    if (waitpid(p, &status, 0) == -1) { perror("waitpid"); return 1; }
    if (WIFEXITED(status))
        printf("Child returned %d\n", WEXITSTATUS(status));
    else if (WIFSIGNALED(status))
        printf("Child killed by signal %d\n", WTERMSIG(status));
    return 0;
}
```

Q9 - argc, argv, envp

Possible task: Show command-line arguments and the HOME entry in envp.

Idea: `main(int argc, char *argv[], char *envp[])` is accepted on typical POSIX/Linux environments.

Compile and run:

```
gcc -Wall -Wextra -g practice/q09_envp.c -o /tmp/practice_eval && /tmp/practice_eval
```

File: `practice/q09_envp.c`

```c
// Mock Q9: inspect argc/argv/envp from main signature in reference sheet.
// gcc -Wall -Wextra q09_envp.c -o q09 && ./q09 hello
#include <stdio.h>
#include <string.h>
int main(int argc, char *argv[], char *envp[]) {
    printf("argc=%d\n", argc);
    for (int i = 0; i < argc; ++i)
        printf("argv[%d]=%s\n", i, argv[i]);
    // envp contains KEY=VALUE strings, ends with NULL.
```

```
    for (int i = 0; envp[i] != NULL; ++i)
        if (strncmp(envp[i], "HOME=", 5) == 0)
            printf("%s\n", envp[i]);
    return 0;
}
```

Q10 - kill and signals

Possible task: Launch `sleep 30`, send SIGTERM, and report how the child terminated.

Idea: `kill()` sends a signal; `waitpid()` reaps the child.

Compile and run:

```
gcc -Wall -Wextra -g practice/q10_signal_kill.c -o /tmp/practice_eval && /tmp/practice_eval
```

File: `practice/q10_signal_kill.c`

```
// Mock Q10: start a child and terminate it with SIGTERM.
// gcc -Wall -Wextra q10_signal_kill.c -o q10 && ./q10
#include <stdio.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <stdlib.h>
int main(void) {
    pid_t p = fork();
    if (p == -1) { perror("fork"); return 1; }
    if (p == 0) {
        char *argv[] = {"sleep", "30", NULL};
        execvp(argv[0], argv); perror("execvp"); _exit(127);
    }
    if (kill(p, SIGTERM) == -1) perror("kill");
    int status;
    if (waitpid(p, &status, 0) == -1) { perror("waitpid"); return 1; }
    if (WIFSIGNALED(status))
        printf("Child terminated by signal %d\n", WTERMSIG(status));
    else if (WIFEXITED(status))
        printf("Child exit code %d\n", WEXITSTATUS(status));
    return 0;
}
```

7. Extra possible instructor modifications (code to insert in your shell)

The examples below are precise changes to **your complete project**, not separate programs.

Q11 - Add `pid` builtin

Task: The `pid` command prints the PID of the shell itself.

1. In `builtins.c` inside `is_builtin()`, add `|| strcmp(name, "pid") == 0` to the returned expression.
2. In `run_builtin()` add this branch before the final return:

```
if (strcmp(cmd, "pid") == 0) {
    if (s->argc != 1) {
        fprintf(stderr, "usage: pid\n");
        return 1;
    }
    printf("Shell PID: %ld\n", (long)getpid());
```

```
    return 0;
}
```

Test: `pid` twice. Both output values should normally be identical for consecutive foreground invocations.

Q12 - Add an `envget` builtin

Task: `envget HOME` displays the value of the environment variable HOME.

1. Add `envget` to `is_builtin()`.
2. Insert into `run_builtin()`:

```
if (strcmp(cmd, "envget") == 0) {
    if (s->argc != 2) {
        fprintf(stderr, "usage: envget NAME\n");
        return 1;
    }
    const char *value = getenv(s->argv[1]);
    if (value == NULL) {
        fprintf(stderr, "%s is not set\n", s->argv[1]);
        return 1;
    }
    printf("%s\n", value);
    return 0;
}
```

Test: `envget HOME`; `envget NOT_REAL_OS_EVAL_VAR` should report unset.

Q13 - Add `setvar NAME VALUE` and `unsetvar NAME` builtins

Add `setvar` and `unsetvar` names to `is_builtin()` and insert:

```
if (strcmp(cmd, "setvar") == 0) {
    if (s->argc != 3) {
        fprintf(stderr, "usage: setvar NAME VALUE\n");
        return 1;
    }
    if (setenv(s->argv[1], s->argv[2], 1) == -1) {
        perror("setenv");
        return 1;
    }
    return 0;
}
if (strcmp(cmd, "unsetvar") == 0) {
    if (s->argc != 2) {
        fprintf(stderr, "usage: unsetvar NAME\n");
        return 1;
    }
    if (unsetenv(s->argv[1]) == -1) {
        perror("unsetenv");
        return 1;
    }
    return 0;
}
```

Test: `setvar EXAMPLE 100`, then `envget EXAMPLE`, then `unsetvar EXAMPLE`.

Q14 - Implement `>>` rather than `>`

Change the output `open` flags from `O_TRUNC` to `O_APPEND` when the parsed operator is `>>`:

```c
int flags = O_WRONLY | O_CREAT;
if (stage->append)
    flags |= O_APPEND;
else
    flags |= O_TRUNC;
int fd = open(stage->outfile, flags, 0644);
if (fd == -1) { perror("open"); return -1; }
if (dup2(fd, STDOUT_FILENO) == -1) {
    perror("dup2");
    close(fd);
    return -1;
}
close(fd);
```

Test: `echo first > result.txt`, `echo second >> result.txt`, `cat result.txt` gives two lines.

Q15 - Redirect a builtin without breaking the prompt

When a **parent builtin** writes to a file, save and restore the shell's standard output:

```c
fflush(stdout);
int saved = dup(STDOUT_FILENO);
if (saved == -1) { perror("dup"); return 1; }

if (redirect_stage(stage) == -1) {
    close(saved);
    return 1;
}

int rc = run_builtin(shell, stage, false);
fflush(stdout);
if (dup2(saved, STDOUT_FILENO) == -1) {
    perror("restore stdout");
    rc = 1;
}
close(saved);
return rc;
```

Note: The full `src/exec.c` contains a stronger version that also handles redirected standard input and cleanup after failures.

Q16 - Reap zombie children (non-blocking)

Inside the main shell loop run:

```c
void reap_jobs(Shell *shell) {
    int status;
    pid_t done;
    while ((done = waitpid(-1, &status, WNOHANG)) > 0)
        remove_job(shell, done);
    if (done == -1 && errno != ECHILD)
        perror("waitpid WNOHANG");
}
```

Never call `waitpid(pid, ..., 0)` immediately for a command that must remain in background.

Q17 - Write a Makefile (complete solution)

```
CC = gcc
CFLAGS = -std=c11 -Wall -Wextra -g
OBJS = main.o parser.o builtins.o exec.o

mysh: $(OBJS)
    $(CC) $(CFLAGS) -o $@ $^

%.o: %.c shell.h
    $(CC) $(CFLAGS) -c $< -o $@

.PHONY: clean
clean:
    rm -f mysh $(OBJS)
```

The recipe indentation must be a TAB. `$@` = target; `$^` = all prerequisites; `$<` = first prerequisite.

Q18 - Write a CMakeLists.txt (complete solution)

```
cmake_minimum_required(VERSION 3.10)
project(mysh C)
set(CMAKE_C_STANDARD 11)
add_executable(mysh main.c parser.c exec.c builtins.c)
target_compile_options(mysh PRIVATE -Wall -Wextra -g)
```

Build with `cmake -S src -B build && cmake --build build`.

Q19 - Static library task

Given `util.c`, make `libutil.a` and link it:

```
gcc -Wall -g -c util.c -o util.o
ar rcs libutil.a util.o
gcc -Wall main.c -L. -lutil -o app
```

Q20 - Shared library task

```
gcc -Wall -fPIC -c util.c -o util.o
gcc -shared -o libutil.so util.o
gcc -Wall main.c -L. -lutil -o app
LD_LIBRARY_PATH=. ./app
```

Practical example files: `practice/libdemo/util.c`, `util.h`, `main.c`, and `build.sh` in the ZIP.

Q21 - Debug a child failing during exec

Compile with `-g`, then start:

```
gdb ./mysh
(gdb) set follow-fork-mode child
(gdb) break run_segment
(gdb) run
```

Run an invalid command from the shell, or put a breakpoint just before `execvp()`. Use `print`, `info locals`, `next`, `step`, and `bt`. Remember the child should report the exec failure and `_exit(127)`; the parent shell must survive.

Q22 - Fix a hanging pipe

Bug: `ls | wc -l` never completes. Typical cause: parent or sibling still holds the writing end of the pipe open, so `wc` never sees EOF.

Solution pattern: After `dup2`, close **both** original pipe descriptors in each child. In parent, close its unneeded read/write descriptors before waiting. See `practice/q04_two_pipes.c`.

8. Final-night preparation plan

30 minutes: Recite what `fork`, `execvp`, `waitpid`, `pipe`, `dup2`, `open`, `close` do. Write the 1-child example without copying.

30 minutes: Write code for `pwd`, `cd`, `exit` and validate `argv[0]`/`argc`. Practice the `pid` builtin modification.

40 minutes: Type the one-pipe two-child example. Explain exactly which process closes which FD. Add `>` or `>>` to child output.

30 minutes: Implement background `&`, `ps`, `kill`, `WNOHANG`, and test with `sleep 30 &`.

20 minutes: Write a Makefile and CMakeLists.txt from memory; practise `gdb` on a small broken example.

If short on time, prioritise: `fork + execvp + waitpid`; `open + dup2`; `pipe + close`; builtins parent vs child; background `WNOHANG`.

Final rule in the lab: Check return values of every meaningful system call, check array bounds, terminate `argv` with `NULL`, close FDs, free memory, reap child processes, and ensure the shell does NOT crash on invalid input.

Good luck with your CS311 evaluation. Use the worked code to learn patterns, then write the solutions from memory.